Statistical and Machine Learning

Intro to R

Mara Lein

Welcome to Statistical and Machine Learning!

- This session provides a shallow overview of R and its most important features.
- We strongly advise you to go through the additional slides of the Statistical Programming Languages course and the provided exercises. You can only learn programming by doing it!
- Information on installing R on your device: HU Box - SML
- Please feel free to message me anytime via email: mara.lein@hu-berlin.de

- Introduction
- R Basics
- R Programming Syntax
- Data Management
- Linear Regression

What is R?

- Programming environment mainly used for data manipulation, calculation and graphical display
- Open source with an active community
- Great variety of packages covering all fields of statistics
- Available for a wide variety of UNIX platforms (incl. Windows and macOS)

Installing and using R

- Available on Comprehensive R Archive Network (CRAN)
- RStudio is recommended as an Integrated Development Environment (IDE) and is also open source
- Code can be stored in files with ending .R
- Comments with #

- Introduction
- R Basics
- R Programming Syntax
- Data Management
- Linear Regression

R Basics: Objects

- An object in R can be (nearly) everything, for example a scalar, a matrix, a function, a model, or a plot
- The objects are assigned to names using <-
- Names are case sensitive!

```
a <- 2          # a is numeric
b <- 3          # b is numeric
a + b
[1] 5
A + b
Error: object 'A' not found
c <- "hello"    # c is character
c
[1] "hello"
```

R Basics: Data types

integer	integer number
numeric	real or decimal number
logical	TRUE or FALSE
character	string of arbitrary characters
factor	categorical groups with fixed levels

```
mode(a)
[1] "numeric"
mode(c)
[1] "character"
```

R Basics: Main structures

vector	one dimensional array of length m, one type
matrix	matrix $m \times n$ arrays of one type
data.frame	tabular data, different data types are possible
list	collection of objects of arbitrary type

```r
data(iris) #load built-in dataset

class(iris) #check object type
[1] "data.frame"

str(iris) #inspect data types, number of obs., variables
'data.frame':   150 obs. of  5 variables:
 $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species     : Factor w/ 3 levels "setosa","versicolor",..:
```

R Basics: help function, load and save data

```
?sin #help for sin-function

install.packages("mvtnorm") #download new package (once!)

library(mvtnorm) #load package to current session

setwd("x:/myRfile") #set working directory
getwd() #get working directory

save.image() #saves workspace to .RData
load(".Rdata") #loads workspace

quit() #quits R
```

R Basics: Accessing Object Elements

- Access individual elements of a vector through brackets [ ]
- Element indexing starts at 1, not 0

```
a <- c(2, 3, 1, 4)

a[3] #third element of vector
[1] 1

a[-3] #all elements except third element
[1] 2 3 4

a[a<3] #filtering based on condition < 3
[1] 2 1
```

R Basics: Accessing Object Elements

- Arrays (e.g. matrices) are also accessed through brackets [ ]
- Dimensions are separated using ,
- A[i, j] gives element in row i at column j of matrix A

```
b <- matrix(1:12, 3, 4)
b
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12

b[2,3]
[1] 8
```

R Basics: Accessing Object Elements

- Lists can also be accessed through brackets [ ]
  - Single square brackets extract a list with the selected elements
  - Double square brackets extract a list element
- Names of list elements can also be directly accessed using the $ operator

```
e <- list("first" = 1, "second" = "two")

str(e[2])
List of 1
 $ second: chr "two"

e[[2]]
[1] "two"

e$second
[1] "two"
```

- Introduction
- R Basics
- R Programming Syntax
- Data Management
- Linear Regression

R Programming Syntax: Functions

- Functions implement tasks you want to do repeatedly and help modularising your code
- Functions may take several parameters (separated by ,)
  - Some parameters have to be specified
  - Functions may have optional parameters
  - Optional parameters have default values

```
myfunction <-  function(x, a = 1) {
  a * sin(x)
}

myfunction(x = pi/2)
[1] 1
```

R Programming Syntax: Conditions

- if - else statements for single logical conditions
- ifelse() function for checking each element of a vector

```
# if-else statement (for scalars)
x <- 1
if(x == 2) {print("x = 2")
  } else{
  print("x != 2")
}
[1] "x != 2"

#if-else function
x <- seq(from = 2, to = -1, by = -1)
x
[1]  2  1  0 -1
sqrt(ifelse(x >= 0, x, NA))
[1] 1.414214 1.000000 0.000000       NA
```

R Programming Syntax: Loops

- Call a function iteratively for different values
- `for`, `while`, and `repeat` structures available
- `apply()` function family
  - Set of highly optimized functions for iterative computations
  - Much faster than standard loops
  - Various versions for matrices, vectors, lists,... exist

`apply()`	Function used on matrices and arrays
`lapply()`	On lists and vectors; returns a list
`sapply()`	Like `lapply()`, returns vector/matrix
`tapply()`	Table grouped by factors
`mapply()`	Multivariate `sapply()`

```r
f <- matrix(1:10, nrow = 2)
f
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    3    5    7    9
[2,]    2    4    6    8   10
# Apply mean function by row
apply(f, MARGIN = 1, FUN = mean)
[1] 5 6

set.seed(123)
c <- list(a = rnorm(10), b = rnorm(2000))
# Apply mean function to each element of list c
lapply(c, mean)
$a
[1] 0.07462564

$b
[1] 0.02842169
```

- Introduction
- R Basics
- R Programming Syntax
- Data Management
- Linear Regression

Data Management: Data Frames

- Analyses often require tabular data of different types
- Data frames are the primary data structure, as they can contain several different data types
- Categorical data are best handled using factor variables
  - Implemented correctly for statistical modeling
  - Very useful in many different types of graphics
  - Correctly identifies the number of degrees of freedom

Data Management: Reading & Writing data

- `R` has already built in data sets (mtcars, iris)
- Read your own dataset for statistical analysis
  - `read.table()`, `write.table()`: very general, for all kinds of text based data
  - `read.csv()`, `write.csv()`: wrappers for `*.table()` for CSV files
  - Connection to Excel, SQL, and other data sources through packages

```
# Read a CSV file with comma separation
data <- read.csv(file)

# Read a CSV file with semicolon separation
data <- read.csv2(file)
```

- Common pitfalls:
  - header = TRUE: Ensure R recognizes the first row as variable names, not data
  - na.strings: Explicitly define characters that represent missing data (e.g., na.strings = c("NA", "-99", " ")) to ensure R treats them as NA

Data Management: Useful R packages

- Reading in and managing your own data can be tedious
- Several packages have been developed to streamline programming
  - `tidyverse`: collection of R packages that simplify data read in, manipulation, plotting, ...
  - `data.table`: provides an alternative to data.frames which is memory efficient and very fast (even for huge data sets)

- Introduction
- R Basics
- R Programming Syntax
- Data Management
- Linear Regression

Linear Regression

- Tries to model relation between dependent variable Y and $1, \ldots, p$ independent variables $X_1, \ldots, X_p$
- Sample of size n is fitted to model with linear effects:

$$y_i = \beta_1 + \beta_2 \cdot x_{2i} + \cdots + \beta_p \cdot x_{pi} + \epsilon_i$$
$$y_i = x_i^T \beta + \epsilon_i$$
$$y = X\beta + \epsilon$$

- Estimate unknown β using least squares (minimizing $\sum \epsilon_i^2$)

Linear Regression: Formula Interface

- Specifies the statistical model based on the columns of a data frame
- Specifies the roles (dependent, independent) as well as transformations of these variables

$y \sim x_1$	Model y as function of x_1
$y \sim x_1 - 1$	Model y as function of x_1 without intercept
$y \sim x_1 + x_2$	Model y as function of x_1 and x_2
$y \sim x_1 \cdot x_2$	Model y as function of x_1 and x_2 and interaction of x_1 and x_2 (short for $y \sim x_1 + x_2 + x_1 : x_2$)

Multiple Linear Regression: Example

- Swiss fertility and socioeconomic indicators (education, infant mortalitiy) from dataset `swiss`
- Model with linear dependence structure:
  $Fertility_i = \beta_0 + \beta_1 Education_i + \beta_2 Infant.Mortality_i + \epsilon_i$

```
data(swiss)
fit <- lm(Fertility ~ Education + Infant.Mortality,
          data = swiss)
summary(fit)
```

```
Call:
lm(formula = Fertility ~ Education + Infant.Mortality, data = swiss)

Residuals:
     Min       1Q   Median       3Q      Max
-15.3906  -6.0088  -0.9624   5.8808  21.0736

Coefficients:
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)       48.8213     8.8904   5.491 1.88e-06 ***
Education         -0.8167     0.1298  -6.289 1.27e-07 ***
Infant.Mortality   1.5187     0.4287   3.543 0.000951 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 8.426 on 44 degrees of freedom
Multiple R-squared:  0.5648,Adjusted R-squared:  0.545
F-statistic: 28.55 on 2 and 44 DF,  p-value: 1.126e-08
```

Linear regression: Useful Functions

```r
# Fitted values of the linear models
fitted(fit) # alternatively: fit$fitted

# Residuals of the linear model
resid(fit) # alternatively: fit$resid

# Coefficients , their variance
coef(fit) # alternatively: fit$coef

# Variance covariance matrix of model
vcov(fit)
```